

KotlinLLM: Leveraging AI for Runtime Logic Delegation in Kotlin

by

Stanislav Sandler

Bachelor Thesis in Software, Data and Technology

Submission: May 27, 2026

Supervisor: Prof. Dr. Timofey Bryksin

Statutory Declaration

Family Name, Given/First Name	Sandler, Stanislav
Matriculation number	30007536
What kind of thesis are you submitting: Bachelor-, Master- or PhD-Thesis	Bachelor Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

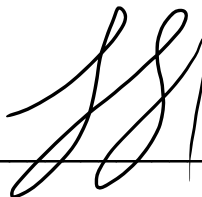
German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

27.05.2026

Date, Signature



Abstract

This thesis presents KotlinLLM, a framework for delegating runtime logic to a Large Language Model (LLM) in Kotlin/JVM applications. KotlinLLM introduces Smart macros, inline utility functions whose behavior is backed by LLM-generated logic that evolves based on the scenarios the application encounters.

The framework is implemented as an IntelliJ IDEA plugin for Kotlin projects targeting the JVM. It provides a public API for Smart macros that is aligned with idiomatic Kotlin conventions. Unlike direct runtime delegation to an LLM service, KotlinLLM stores generated behavior as Kotlin source code. The plugin then manages the runtime evolution loop: it prepares templates for storing generated Kotlin code, launches the application under the Java Debug Interface (JDI), intercepts unsupported Smart macro inputs that trigger evolution, provides context about the actual runtime values and application types, compiles the generated update, and redefines the affected class in the running JVM.

We also propose an approach for designing the LLM agent used in the regeneration loop and describe the limitations and challenges we encountered. We validate the approach on a synthetic GitHub Issue Radar application and an adapted version of the Spring Petclinic Kotlin application. The experiments show that Smart macros can generate Kotlin code, evolve from runtime scenarios, and add low overhead once the logic has been generated by the LLM.

Acknowledgments

I would like to thank JetBrains for supporting this research and JetBrains employees for guiding me through this thesis. In particular, I would like to thank Anastasiia Birillo for being my mentor and setting this project up. I would also like to thank Simon Ogorodnik and Marat Akhin for helpful brainstorming sessions and for sharing my excitement during the making of KotlinLLM.

Contents

1	Introduction	1
2	Existing Approaches and Motivation	2
3	Contributions	2
4	Motivating Example: GitHub Beginner Issue Radar	3
4.1	Parsing Repository Data	3
4.2	Mocking the GitHub Service	5
5	Design Requirements	6
5.1	Idiomatic and Explicit Calls	6
5.2	Reliable Dispatch	6
5.3	Compilable Empty State	7
5.4	Detecting Missing Cases	7
5.5	Portable Generated Code	7
6	Architecture and Framework Design	7
6.1	Public API	8
6.2	Static Infrastructure	9
6.3	Dynamic Infrastructure	9
6.3.1	Providers	9
6.3.2	Logic Containers	10
7	Runtime Evolution with the IntelliJ Plugin	12
7.1	Pre-Launch Project Analysis	12
7.2	Generated Source Preparation	12
7.3	Application Launch and Debug Setup	13
7.4	Runtime Interception	13
7.5	Agent Update and Class Redefinition	14
8	LLM Agent	14
8.1	Why an Agent Is Needed	14
8.2	Initial Tool Set	14
8.3	Avoiding Overgeneralization	14
8.4	Managing Context Size	15
8.5	Exploring Runtime Input	15
8.6	Target Type Graph Construction	15
8.7	Final Tool Set	16
9	Validation	16
9.1	Methodology and Metrics	17
9.2	Spring Petclinic Kotlin	18
9.3	GitHub Issue Radar	19
10	Limitations	21
10.1	Platform Constraints	21
10.2	LLM Reliability and Security	21
10.3	Launch Configuration	22

10.4 Type and Mock Support	22
11 Conclusion	23
12 References	23

1 Introduction

The primary application of Large Language Models (LLMs) in software engineering centers on code generation and program comprehension [14]. However, utilizing LLMs to dynamically alter the active runtime of compiled applications remains underexplored.

A motivating example can be taken from agentic LLM workflows. In such workflows, an LLM is not used only for a single completion request; instead, it acts as an agent that can inspect project files, call tools, run commands, observe results, and iteratively decide what to do next. These agents may sometimes inject observability logic to inspect an application's real-time behavior and state. Recent literature, such as the MTP Framework, provides empirical evidence that LLMs produce higher-quality code when they have access to the runtime context [3]. This is also supported by print-debugging work, where execution traces and log observations help LLMs repair generated code more effectively [6].

To provide runtime context seamlessly, we propose KotlinLLM, an approach that lets developers interact with the LLM directly from application code. This keeps the LLM integration explicit, persistent and portable. Explicit means that it is clear at the call site that the developer is using an LLM-backed feature, which makes the behavior visible during code review. Persistent means that learned behavior is stored as generated Kotlin code instead of remaining only in the runtime session. Portable means that this generated code can be distributed with the project, for example through ordinary version control systems.

Although Kotlin supports multiple compilation targets, redefining application logic at runtime is not a platform-agnostic capability. For Kotlin applications running on the JVM, the most straightforward mechanism is JVM class redefinition, which allows tooling to replace the bytecode of an already loaded class while the process is running [10]. Consequently, this thesis focuses the KotlinLLM design specifically on Kotlin/JVM projects.

The features KotlinLLM introduces are called Smart macros. A Smart macro is a regular Kotlin function call whose behavior is backed by dynamically generated Kotlin code. At initialization, the backing code is a no-op stub, but when the running program reaches a scenario where the Smart macro is used, KotlinLLM can suspend execution, collect the actual runtime values, and use the LLM agent to generate an update for the Smart macro designed for handling this scenario. This code is then hot-reloaded into the running JVM. The macro is “smart” because its behavior evolves from concrete runtime usage instead of being fully specified before the program starts. It remains a macro-like language feature because the call site stays compact and explicit, hiding the complex implementation details.

We chose Kotlin as the programming language to develop this integration in. Kotlin's Domain-Specific Languages (DSL) support allows creating clean, idiomatic language constructs that mimic native language keywords and minimize cognitive friction for developers. Furthermore, the Kotlin tooling ecosystem within IntelliJ IDEA, primarily the access to the Program Structure Interface (PSI), provides an optimal platform for orchestrating this runtime integration [7].

Implementing a plugin for IntelliJ gives us enough control over the application's flow, while avoiding more invasive approaches such as a Kotlin compiler plugin. This approach allows for quicker prototyping and flexibility.

The rest of this thesis introduces KotlinLLM, explains how Smart macros are represented in Kotlin code, describes how the IntelliJ plugin evolves their implementations at runtime, and validates the approach on two Kotlin projects.

2 Existing Approaches and Motivation

The idea of calling an LLM during application execution is relatively new and underexplored.

For example, `byLLM` is a Python extension for delegating application logic to the LLM [3]. In other words, it allows functions, classes, and methods to be backed by the LLM. Every call to this delegated logic triggers a call to an LLM, with additional context from the relevant values as well as the graph representing the type dependencies. The idea is to produce output that respects the expected type. To use `byLLM`, the developer only needs to use the `'byLLM'` keyword. The rest of the context is resolved based on the identifier names.

Similar to `byLLM`, `nightjar` is a Python extension that enables inline prompts in the language [2]. LLMs have full access to the program's state, variables, heap, and control flow. Instead of using a keyword, `nightjar` requires the developer to write a prompt explaining the expected behavior.

The first two approaches delegate application logic to the LLM. Healer introduces a different approach. It focuses on achieving self-healing applications. On failure, the LLM inspects the stack trace, the application state, and execution environment to attempt to recover the current state [13].

KotlinLLM takes inspiration from these approaches by creating a DSL for LLM features as in `byLLM` and `nightjar`, while also adding new persistent logic to the application as in Healer. The current solutions mostly work on interpreted languages like Python. We, however, wanted to target the Kotlin language.

Persistence is desirable because it turns an LLM-assisted runtime update into ordinary project code. Once the Smart macro implementation is generated, it can be distributed with the project, reviewed, tested, and verified like other Kotlin source files. This also improves reliability and reproducibility: the same input scenario does not depend on a future LLM call, and the generated behavior can be inspected before the application is shipped.

Since Kotlin is not an interpreted programming language, we can alter code at runtime only using JVM class reloading. This introduces limitations since we cannot add or remove any fields or methods, or add new classes.

3 Contributions

Since we opted to make the LLM-to-application interaction a language feature, we prioritize persistence, explicitness and portability. Persistence differentiates our approach from existing solutions discussed in Section 2. Portability is also central to the design: a project that uses KotlinLLM should not become dependent on the IntelliJ plugin to run. The generated code remains ordinary Kotlin code and can execute without the plugin, while the plugin is only needed for the runtime evolution of Smart macros.

We call the KotlinLLM features Smart macros: persistent, portable, evolving functions that autonomously adapt their logic when their current implementation fails to process an unseen scenario.

From the developer's side, a call to Smart macros is:

- persistent, since the Smart macro logic is stored as Kotlin code,
- explicit, since it is a clear function call,
- portable, since generated behavior can run without the KotlinLLM plugin,
- flexible, as the Smart macro logic updates when encountering a scenario that is not yet covered.

Specifically, we present two Smart macro implementations:

- `asLlm<F, T>`: A Smart macro that expands into a type-safe parser. Its primary use case is transformation of unstructured and semi-structured data (F) into strictly typed Kotlin data classes, enums or primitive types (T) at runtime.
- `mockLlm<T>`: A Smart macro that expands into a stateful mock implementation of the requested interface (T). Its behavior and state adapt based on the specific methods invoked and the actual runtime values passed to it.

4 Motivating Example: GitHub Beginner Issue Radar

The motivating example used throughout the thesis is a small utility application called GitHub Beginner Issue Radar. The application asks the user for a GitHub repository and returns a list of issues that are likely suitable for beginners.

The problem looks simple at first. A user provides a repository, the application calls the GitHub API, and the response is converted into a list of domain objects. The application uses the following small model and service boundary:

```
data class Issue(  
    val title: String,  
    val url: String,  
    val labelNames: List<String>,  
    val isBeginnerFriendly: Boolean  
)  
  
interface GithubService {  
    fun fetchAllIssues(issuesApiUrl: String): String  
}
```

4.1 Parsing Repository Data

There are two places where the application has to deal with input that is easy for a human to understand, but inconvenient to encode as fixed application logic. The first one is the conversion from a repository input to the corresponding GitHub REST API URL. During development and testing, the input may appear in several formats:

- `https://github.com/JetBrains/kotlinconf-app` – a full repository URL copied from the browser.
- `github.com/JetBrains/kotlinconf-app` – the same repository URL without the protocol prefix.
- `JetBrains/kotlinconf-app` – GitHub's owner-and-repository shorthand.
- `https://github.com/JetBrains/kotlinconf-app/issues` – an issues page URL instead of the repository root.

This input is then used to create a correct URL for GitHub's REST API [4]. The key motivation is that the variance in the input does not change the overall goal: get the API endpoint. Using a Smart macro here allows the developer to remain one level of abstraction higher, since handling URL normalization edge cases is delegated to KotlinLLM. The `hint` parameter is necessary because the type information alone is ambiguous in this case: the Smart macro receives a `String` and must produce another `String`, but the desired transformation is specifically repository input normalization into a GitHub issues API URL.

```
val issuesApiUrl: String = asLlm(
    repoInput,
    hint = "github API url: get all issues, including closed issues"
)
```

For example, if `repoInput` refers to the KotlinConf app issue page, this Smart macro should normalize it to the corresponding GitHub API endpoint and include both open and closed issues [4]. By contrast, a regular Kotlin implementation would fail to parse an input the developer did not cover beforehand, and would require the developer's attention to support the new case, as well as rerun the application.

The second example is the parsing of the API response. GitHub API returns the response body as a `String`, while the rest of the application wants a typed list of `Issue` values. The response already contains API-shaped data, including issue titles, URLs, `labelNames`, and repository-specific naming conventions, but the semantic rule for beginner friendliness depends on each repository:

```
val issues: List<Issue> = asLlm(
    response, // [{issue}, {issue}, {issue}, ...]
    hint = """
        Sets isBeginnerFriendly based on labelNames.
        Unique approach for each repository.
        """
)
```

The `hint` parameter is required here for a different reason than in URL normalization. The source and target types tell the model that it should parse a `String` into a list of `Issue` values, but they do not express the non-trivial insight that each repository may require a unique approach and that beginner-friendly `labelNames` differ across repositories. The hint gives the LLM this semantic intent explicitly. The complete application workflow composes these two Smart macros around the ordinary GitHub service call:

```
class GithubIssueRadar(
    private val githubService: GithubService
) {
```

```

fun loadIssues(repoInput: String): List<Issue> {
    val issuesApiUrl: String = asLlm(
        repoInput,
        hint = "github API url: get all issues, including closed issues"
    )

    val response = githubService.fetchAllIssues(issuesApiUrl)

    return asLlm(
        response,
        hint = """
            Sets isBeginnerFriendly based on labelNames.
            Unique approach for each repository.
            """
    )
}

```

The Smart macro should not solve every possible GitHub labeling convention in one attempt. Instead, KotlinLLM treats each unsupported runtime input as a new scenario. When the application encounters a repository whose `labelNames` values are not covered yet, the generated parser can evolve. If the same repository, or the same label-name convention, appears again later, the application reuses the generated Kotlin code instead of asking the LLM again. To update the Smart macro's logic, the application does not have to restart. New logic is recompiled at runtime and injected into the already running process.

4.2 Mocking the GitHub Service

The same example also motivates `mockLlm`. Tests might need `GithubService` to return realistic API-shaped data without making a network call. With KotlinLLM, the test can ask for an explicit generated mock and use it as a dependency of the component being tested:

```

@Test
fun loadsIssuesWithoutCallingGithubApi() {
    val mockService = mockLlm<GithubService>()
    val radar = GithubIssueRadar(mockService)

    val beginnerOnly = radar.loadIssues(
        "https://github.com/example/beginner-only-issues"
    )
    val difficultOnly = radar.loadIssues(
        "https://github.com/example/difficult-issues-repo-only"
    )

    assertTrue(beginnerOnly.isNotEmpty())
    assertTrue(beginnerOnly.all { it.isBeginnerFriendly })
    assertTrue(difficultOnly.isNotEmpty())
    assertTrue(difficultOnly.none { it.isBeginnerFriendly })
}

```

In this test, the goal is not to test the generated mock itself. The goal is to test the application code that depends on `GithubService`, while avoiding a network call. The repository names are intentionally artificial: one should produce only beginner-friendly issues, while the other should produce none. The generated mock keeps the setup lightweight and inspectable, while still using API-shaped data instead of arbitrary dummy values.

GitHub Issue Radar demonstrates the reason for introducing Smart macros as explicit language-level calls. The code remains compact and typed, the generated behavior is persistent, and runtime values provide the context needed to handle input formats, repository-specific semantics, and lightweight test doubles.

5 Design Requirements

The design requirements are derived from the motivating example and from the constraints of applying runtime LLM evolution to a compiled Kotlin application. GitHub Issue Radar shows that Smart macros must be explicit at the call site, route each call to the correct generated behavior, start from valid code before any runtime scenario is known, detect unsupported runtime inputs, and preserve learned behavior as ordinary project code. The JVM setting adds another constraint: generated classes must already exist and compile before the application starts, because runtime evolution can redefine loaded classes but cannot freely introduce new program structure after launch.

These observations define the requirements below. They are not intended as a general taxonomy of LLM programming systems. Instead, they are the minimum requirements needed for KotlinLLM to support persistent runtime adaptation in Kotlin without turning the application into an opaque dependency on an LLM service.

5.1 Idiomatic and Explicit Calls

The developer's interaction with Smart macros should happen through an idiomatic Kotlin API. Smart macro calls should remain explicit at the call site, but they should not add unnecessary noise to the codebase or require developers to manage generated logic containers manually. In particular, it must be clear to the developer and to code reviewers that the call uses LLM-generated behavior, instead of looking like an ordinary hand-written helper whose implementation is fully known in advance.

This requirement follows from the goal of letting developers interact with the LLM from application code. The call site should make LLM-backed behavior visible during review, while still looking like ordinary Kotlin code. This is important because generated behavior can change as new runtime scenarios are encountered, so hiding it behind an implicit transformation would make the source code harder to reason about.

5.2 Reliable Dispatch

The design must guarantee that every Smart macro call resolves to the appropriate generated implementation. Dispatch must account for the Smart macro kind and the relevant type information, including cases where several calls share the same source type but differ in target type.

This requirement follows directly from the motivating example. One Smart macro normalizes repository input from `String` to `String`, while another parses an API response from

`String` to `List<Issue>`. The runtime must route each call to the corresponding generated container; otherwise an update for one scenario could affect a different Smart macro.

5.3 Compilable Empty State

The project must remain compilable during every stage of a Smart macro's evolution. In particular, every generated Smart macro implementation must have an empty but valid initial state before any scenario-specific logic has been generated.

This requirement comes from the compiled JVM setting. KotlinLLM needs the generated classes to exist before launch so the application can load them and JDI can later redefine them. Therefore, even an untrained Smart macro must be represented by valid Kotlin source.

5.4 Detecting Missing Cases

The design must provide a mechanism for detecting when generated Smart macro logic does not cover the current runtime input. In that case, the generated implementation should not silently return an incorrect result. It should reach a well-defined path that can trigger Smart macro evolution.

This requirement follows from the runtime-learning goal. KotlinLLM can only evolve a Smart macro when the generated code can recognize that the current input is unsupported. Without this path, the system would either silently return wrong data or fail without the precise runtime context needed for an update.

5.5 Portable Generated Code

A project generated with KotlinLLM should remain a normal Kotlin project. In other words, the set of setups that can run a Kotlin project should be equal to the set of setups that can run the same project after KotlinLLM has generated code for it, assuming the same Kotlin and dependency versions are available.

The portable form is allowed to execute only the generated behavior it already contains. If an unseen runtime case appears, the generated code should still fail through the defined missing-case path, but it does not need to learn a new case unless the KotlinLLM evolution tooling is available.

This requirement follows from persistence as a central design goal. Learned Smart macro behavior should be reviewable, testable, distributable, and executable as ordinary Kotlin code, instead of depending on a future LLM call for the same already-seen scenario.

6 Architecture and Framework Design

The KotlinLLM solution follows from the requirements in Section 5. The framework consists of three parts:

- Public API
- Static infrastructure
- Dynamic infrastructure

The public API contains the visible Smart macro entry points, such as `asLlm` and `mockLlm`. Static infrastructure contains the manager classes that route public API calls into generated code. Dynamic infrastructure is created and updated by the plugin during application runtime; it contains generated providers and logic containers for the encountered scenarios.

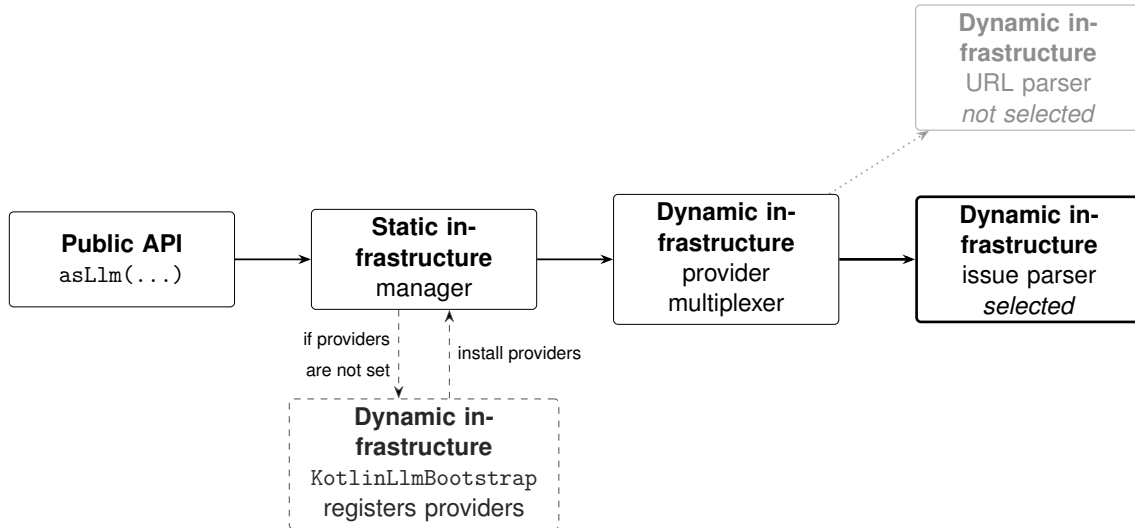


Figure 1: Request flow from a public Smart macro call through the provider multiplexer to the selected generated logic container, including the optional provider registration path.

6.1 Public API

The public API is distributed as part of `KotlinLlm.kt`. This file contains the stable API that is not generated by the plugin. The entry points to the Smart macros are inline functions named after the Smart macro types. This supports Requirement 5.1, because inline functions allow hiding code under a label, without runtime costs, and lead to idiomatic and keyword-like usage.

```

inline fun <reified From, reified To> asLlm(from: From, hint: String = ""): To {
    val parser = AsLlmManager.resolve(
        typeOf<From>(),
        typeOf<To>()
    ) as? AsLlmParser<From, To>
    ?: error("No asLlm parser for ${typeOf<From>()} -> ${typeOf<To>()}")
    return parser.parse(from, hint)
}
  
```

The public API exposes the language-level surface of Smart macros. It does not contain generated scenario logic itself. Instead, each Smart macro inline function expands to a call into the static infrastructure. In GitHub Issue Radar, this means that the application can keep calls such as `asLlm<String, String>(repoUrl)` and `asLlm<String, List<Issue>>(response)` directly inside the workflow. The calls are explicit, but the developer does not need to manage generated parser classes manually.

6.2 Static Infrastructure

Static infrastructure contains manager classes – `AsLlmManager` and `MockLlmManager`. These classes are distributed with the project and are not regenerated by the plugin. Their main purpose is to bridge the stable public API and the generated providers.

A regular Kotlin reference from the public API to the generated providers would make the project non-portable. When the public API and static infrastructure are distributed, the generated providers may not exist yet, so code that directly imports or references them would fail to compile. Instead, the managers load a known generated bootstrap class reflectively:

```
Class.forName("com.jetbrains.kotlinllm.generated.KotlinLlmBootstrap")
```

The manager expects the plugin to generate this bootstrap class. The bootstrap object itself does not contain dispatch or parsing logic. Its role is only to register the generated providers with the static managers when the class is loaded:

```
public object KotlinLlmBootstrap {  
    init {  
        AsLlmManager.install(KotlinLlmGeneratedAsLlmProvider)  
        MockLlmManager.install(KotlinLlmGeneratedMockLlmProvider)  
    }  
}
```

Loading this class runs the initializer, which installs the generated `asLlm` and `mockLlm` providers into the static managers. After that, the managers forward Smart macro calls to the registered providers.

This bridge supports Requirement 5.5. The public API and static infrastructure remain compilable and distributable because they have no compile-time references to generated providers. The plugin can still attach dynamic infrastructure when it is available.

6.3 Dynamic Infrastructure

Dynamic infrastructure consists of generated Smart macro files created and maintained by the KotlinLLM plugin. The term dynamic means that these files are part of the project source tree, but their contents are expected to change as KotlinLLM encounters new runtime scenarios. They are generated before the target application starts, and later the plugin may update their method bodies through runtime evolution. This is how Smart macros remain flexible: the public API and manager classes stay stable, while the generated providers and logic containers evolve with the observed application behavior.

6.3.1 Providers

The first generated type is the `Provider` class. This class contains the multiplexing logic for Smart macros and provides the routing mechanism required by Requirement 5.2. The `Provider` class is essentially a pattern matcher that returns the logic container responsible for a specific Smart macro call. For `asLlm`, the provider compares the runtime `KType` values of the source and target types. If the pair matches one of the generated cases, it returns an `AsLlmParser`. For `mockLlm`, the provider compares the requested interface type and returns a generated mock object.

This dispatch structure supports projects with several Smart macros that share a source type but differ in target type. In GitHub Issue Radar, one `asLlm` call converts a repository input into a GitHub API URL, while another converts a GitHub API response into `List<Issue>`. Both calls start from `String`, but the provider routes them to different generated logic containers by comparing the full type information.

A simplified provider for the GitHub Issue Radar parsers has the following shape:

```
object GeneratedAsLlmProvider : AsLlmProvider {
    override fun resolve(from: KType, to: KType): AsLlmParser<*, *>? {
        return when {
            from == typeOf<String>() &&
                to == typeOf<List<Issue>>() ->
                    StringListIssueParser()

            from == typeOf<String>() &&
                to == typeOf<String>() ->
                    StringStringParser()

            else -> null
        }
    }
}
```

Type matching in providers is not always direct. In practice, comparing Kotlin types is less direct than the simplified provider suggests. For example, when Kotlin code interoperates with Java, a method may expose a platform type. Such a type, written as `T!`, does not reliably match either `T` or `T?` when the provider compares values against `typeOf`. Direct equality can therefore reject a case whose classifier and type arguments are otherwise correct.

For this reason, generated providers use a structural matching utility instead of plain `KType` equality. The utility compares classifiers and generic arguments recursively, while avoiding dispatch failures caused only by nullability or platform-type differences. This supports the reliable dispatch goal from Requirement 5.2.

```
private fun typesMatch(first: KType, second: KType): Boolean {
    if (first.classifier != second.classifier) return false
    if (first.arguments.size != second.arguments.size) return false

    return first.arguments.zip(second.arguments).all { (arg1, arg2) ->
        val type1 = arg1.type
        val type2 = arg2.type
        if (type1 == null || type2 == null) return@all type1 == type2
        typesMatch(type1, type2)
    }
}
```

6.3.2 Logic Containers

The second generated type is the logic container. This class contains the actual behavior for the Smart macro.

For `asLlm<F, T>`, this class implements the `AsLlmParser<F, T>` interface, which has a single method `parse` that returns the value of type `T`. Calling `asLlm<F, T>` actually resolves to a call to this method.

A simplified generated parser implementation has the following structure:

```
class StringListIssueParser : AsLlmParser<String, List<Issue>> {
    override fun parse(from: String, hint: String): List<Issue> {
        val result = parse_(from)
        if (result != null) return result.value

        parseRegenerate(from, hint)
        error("No generated case matched")
    }

    private fun parse_(from: String): ParsedResult? {
        if (isKotlinConfAppResponse(from)) {
            return parseKotlinConfAppResponse(from)
        }
        return null
    }

    private fun parseRegenerate(from: String, hint: String) = Unit

    private data class ParsedResult(val value: List<Issue>)
}
```

This skeleton supports Requirement 5.3. The GitHub Issue Radar parser for `List<Issue>` exists before it knows how to parse a concrete GitHub response. It still compiles because the generated class already contains the public method, the private implementation body, the regenerate hook, and the wrapper type used for successful results. The initial private implementation can simply return `null`, which means that no generated case has matched yet.

The private implementation body and regenerate hook also support Requirement 5.4. If GitHub Issue Radar receives a response from a repository whose `labelNames` values are not covered yet, the generated parser should not silently return an incorrect result. Instead, the private implementation returns `null`, the public method reaches the stable regenerate hook, and KotlinLLM can inspect the runtime response before generating a new case.

For `mockLlm<T>`, this class implements the interface `T`. Calling `mockLlm<T>` actually resolves to initialization of the mock object.

Unlike `asLlm` parsers, generated mocks may need to preserve state across method calls. A parser usually receives one input value and returns one output value. A mock can represent a stateful dependency, such as a database or repository. In that case, write-like method calls should affect the result of later read-like method calls.

Generated mock classes therefore contain a state map:

```
private val state: MutableMap<Any?, Any?> = mutableMapOf()
```

The generated method bodies can use this map to store observations and synthetic data

accumulated so far. For example, a generated database mock can keep inserted entities in `state`, and later return them from query methods. This makes `mockLlm` behavior depend not only on the current method arguments, but also on the interaction history of the mock object.

In the GitHub Issue Radar project, the provider classes stay simple. Their responsibility is only routing. The generated parser and mock classes contain the changing behavior.

Storing learned Smart macro behavior as Kotlin source code supports Requirement 5.5. If the GitHub Issue Radar parser has already learned how to parse the KotlinConf app repository response, the parser can be compiled, tested, distributed, and executed on another setup without requiring KotlinLLM-specific runtime evolution. It behaves like ordinary project code. The limitation is that a portable setup can only execute the generated behavior it already has; if a new repository uses an unseen label-name convention, the generated code still follows the missing-case path described by Requirement 5.4.

7 Runtime Evolution with the IntelliJ Plugin

Runtime evolution is the responsibility of the IntelliJ plugin. This keeps the implementation lightweight: the plugin can prepare generated files, control the application launch, and observe runtime values without changing the Kotlin compiler. It prepares generated source code before launch, starts the target application under JDI [10], intercepts generated regenerate hooks, asks the LLM agent for a narrow update, and redefines the changed class in the running JVM. This section follows those stages and uses GitHub Issue Radar only where a concrete example clarifies the mechanism.

7.1 Pre-Launch Project Analysis

Before the target application starts, the KotlinLLM run configuration prepares the generated source folder. If the project was not initialized yet, the plugin creates the generated package, the bootstrap files, and the provider files. Then `AsLlmMode` and `MockLlmMode` scan the indexed Kotlin project for Smart macro calls.

The plugin uses IntelliJ PSI and Kotlin analysis to recover the types involved in each call. For `asLlm`, it records the source and target types. In GitHub Issue Radar, this produces requests such as `String -> String` for repository URL normalization and `String -> List<Issue>` for API response parsing. For `mockLlm`, the plugin records the requested interface type, such as `GithubService`.

7.2 Generated Source Preparation

The analysis stage produces the generated files needed for the run. For GitHub Issue Radar, setup creates the logic containers for the `String to String` and `String to List<Issue>` calls, as well as `GithubServiceMock`. The `String to String` logic container parses user repository input into a GitHub API URL, while the `String to List<Issue>` logic container parses the API response into issue objects. Setup also creates the bootstrap and provider files required to connect the public Smart macro calls to those generated classes at runtime.

At this point the generated classes may still contain only empty behavior. The important runtime property is that the files exist and compile before the target application starts.

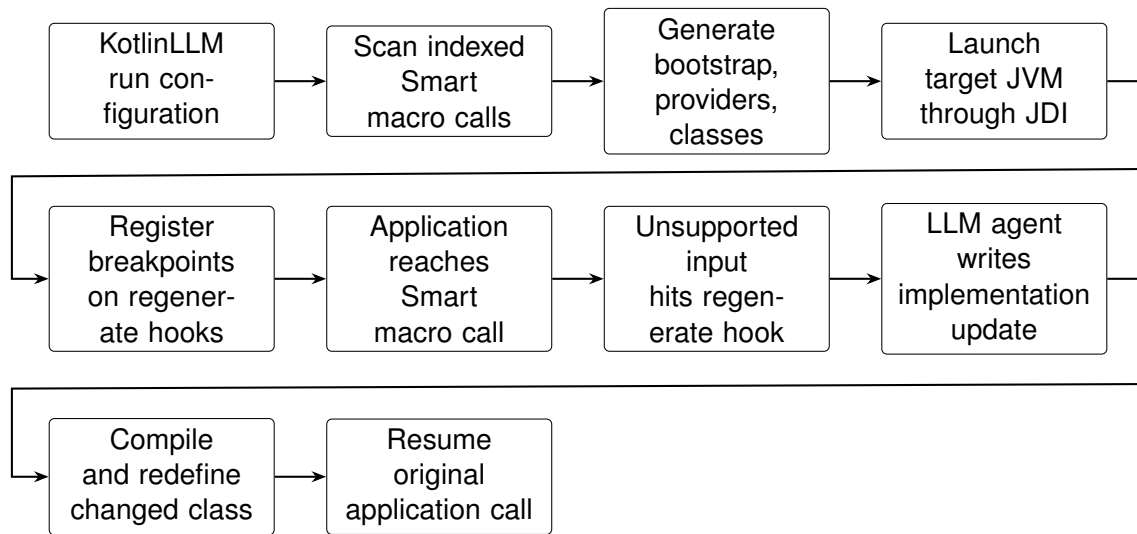


Figure 2: Runtime flow when a project is launched through the KotlinLLM run configuration.

This gives JDI a stable set of classes and methods to redefine later, after the application reaches an unsupported Smart macro scenario.

7.3 Application Launch and Debug Setup

After generation, the plugin launches the original run configuration through JDI [10]. The target program still runs as a normal JVM application, but KotlinLLM also owns the debug connection. This lets KotlinLLM use the same pre-run steps, classpath, JVM flags, and working directory as the developer’s normal IntelliJ run configuration.

The plugin registers breakpoints on every generated regenerate hook. In JDI terms, these are ordinary breakpoint requests on known methods, with the suspend policy set to suspend only the event thread. Suspending only the event thread is enough for KotlinLLM to inspect the current Smart macro call while reducing interference with the rest of the process.

7.4 Runtime Interception

When the application reaches a Smart macro call, the call is routed to the generated implementation prepared for that call site. In GitHub Issue Radar, the repository URL macro reaches the generated string-to-string parser, while the API response macro reaches the generated issue parser.

If the current generated implementation handles the input, the application continues normally. If no generated case matches, execution reaches the corresponding regenerate hook. This interception point exists because the design explicitly requires missing cases to take a well-defined path, as described in Requirement 5.4, and because the logic container skeleton in Section 6.3.2 routes unmatched inputs to a regenerate no-op. The hook itself is a no-op; its purpose is to provide a stable breakpoint location. When the breakpoint is hit, KotlinLLM reads the suspended frame to capture the actual arguments, the hint, and the relevant type information. Runtime values are embedded into the LLM agent prompt through their JVM string representation: KotlinLLM relies on the fact that

JVM objects provide a `toString()` method, so the captured value can be presented to the model as text.

7.5 Agent Update and Class Redefinition

Once the hook is intercepted, KotlinLLM calls the LLM agent. The agent receives tools for targeted inspection rather than the whole project. It can read captured runtime values, grep inside large inputs, inspect the target type graph described in Section 8.6 through `readActualType`, and submit a narrow implementation update.

KotlinLLM inserts the update into the generated source file, rebuilds that file, collects the new class bytes, and calls the JDI class redefinition API.

After redefinition, the plugin reinstalls breakpoints for the redefined class and resumes the original application call. The call is retried and reaches the updated generated implementation. If the generated guard matches the runtime value, the application receives the expected Kotlin value and continues without restarting.

8 LLM Agent

8.1 Why an Agent Is Needed

To update Smart macro logic, we use LLMs. It is worth noting that it is not optimal to approach this task as a single LLM call. When dealing with large input values or complex result types, we require a large number of tokens for each request. We also expect some exploratory work on the input values so that the LLM can get better context. This is why we design an LLM agent, equipped with tools to analyze the input values and types. This follows the general view of agentic LLM systems as models that reason, act through tools, and interact with their environment [11].

8.2 Initial Tool Set

The agent was initially equipped with the minimal set of tools required to upgrade the existing Smart macro:

- Read the values passed to the Smart macro
- Submit the generated code

This design is enough to make a first update, but it gives the agent very little support for writing reusable and robust code.

8.3 Avoiding Overgeneralization

For instance, a tool could be added to read the current Smart macro implementation. This would reduce code repetition and increase robustness, since the agent could reuse existing valid code.

However, full read access can distract the agent from the unsupported runtime case that triggered regeneration. If an existing generated case looks similar to the new input, the model may decide that the old case is already general enough, or it may broaden that

case until it accidentally changes previous behavior. This risk grows as the Smart macro implementation becomes longer, because existing code starts to dominate the context.

To address this, we provide a Utility repository instead of direct read access to the current Smart macro implementation. The agent may decide that it needs some utility function to write code. It can register, delete and browse utility functions, and use them in the resulting updated code. This keeps reusable helper logic available, while keeping the agent focused on the unsupported scenario that triggered the update.

8.4 Managing Context Size

Another issue is that in a real project, the values, types, and the Smart macro logic itself can take up a large part of the LLM's context. KotlinLLM addresses this by adding pagination to the tools output. The model can list the tool output page by page, reducing the expected amount of tokens consumed.

8.5 Exploring Runtime Input

The LLM also does not always need to read the input continuously. When the model attempts to deduce the scenario, it essentially explores the input. To help this exploration be more efficient, the model is provided with a grep tool.

The inspiration is taken from the way CLI agents such as Claude Code and Codex perform exploration [1, 9]. This tool proved to be useful and is the most called in the validation experiment.

8.6 Target Type Graph Construction

The LLM agent needs more than the raw runtime value. It also needs to know the shape of the type it must produce. Related LLM-integrated programming systems similarly use type information to constrain LLM-backed behavior [3]. KotlinLLM builds this information from IntelliJ indexing and Kotlin type resolution before exposing it through the agent's `readActualType` tool.

Internally, KotlinLLM represents resolved Kotlin types as `ActualType`. An `ActualType` stores the type name, package name, type arguments, constructor parameters, constructor text, and nullability. The conversion uses Kotlin analysis when possible; when the type is a project class, the plugin records constructor parameters from the primary constructor. For data classes and enums, the constructor or enum declaration is also retained so the model can see how values should be created.

The rendered type graph is built by recursively collecting unique dependencies. The traversal starts at the target type, then follows type arguments and constructor parameters. The `render()` method prints each dependency with its fully qualified name. If the dependency is a non-Kotlin type with constructor text, that constructor is printed. If the dependency has constructor parameters, the output also lists the types it needs.

The same type may be reached through several fields or type arguments, and KotlinLLM keeps it as a single dependency. In the GitHub Issue Radar example, `String` is needed for `title`, `url`, and the element type of `labelNames`. Treating these as one dependency avoids presenting the agent with repeated type descriptions and makes the generated source easier to prepare.

The graph is also prepared before the application is launched. Later, during hot reload, the LLM is expected to update only prepared implementation bodies. It should not require new imports, because the generated class header is treated as stable after the target JVM has loaded the class. By collecting the full graph up front, KotlinLLM can import the project classes and nested dependency types that the generated code may need.

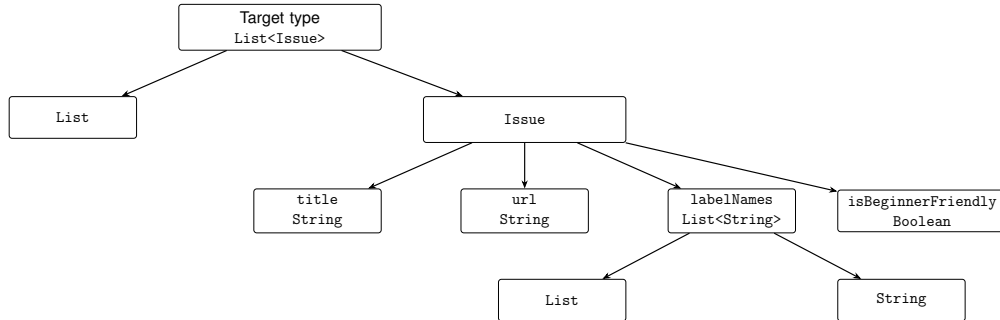


Figure 3: Type dependency graph built for the GitHub Issue Radar `List<Issue>` target type, where `Issue` is a project data class.

In GitHub Issue Radar, this graph tells the agent that the response parser must produce a list of `Issue` values, and that each `Issue` contains fields such as `title`, `url`, `labelNames`, and the derived `isBeginnerFriendly` flag. This keeps the prompt focused on the actual output contract instead of requiring the whole project source to be copied into the LLM context.

8.7 Final Tool Set

Summing up, the final toolset of the LLM agent is as follows:

- `readRuntimeValue`: reads the runtime value captured at the regenerate hook.
- `grepRuntimeValue`: searches large captured values page by page.
- `readActualType`: reads the rendered target type graph described in Section 8.6.
- Utility repository tools: `registerUtilityFunction`, `deleteUtilityFunction`, `listUtilityFunctions`, and `readUtilityFunction`
- `submitImplementation`: submits the generated implementation update for the unsupported case.

9 Validation

KotlinLLM is validated on two Kotlin projects. Each project stresses a different part of the system: GitHub Issue Radar exercises unstructured data parsing and scenario detection, while Spring Petclinic Kotlin exercises a larger application flow with repeated `asLlm` use.

For these projects, we also measured the effect of the KotlinLLM run configuration on application start time, and the runtime overhead when Smart macros do not update their logic. The project-specific sections below focus on the more interesting validation target for each project.

9.1 Methodology and Metrics

The validation is based on structured logs emitted by the KotlinLLM plugin during each run. Every log entry contains a timestamp and an event type; operation events also record their duration in `durationMs`, and LLM request events record the number of tool calls in `count`. The event names below correspond to the runtime-flow entries in Figure 2:

- **KotlinLLM run configuration:** `run_started` marks the start of the plugin-owned launch flow, and `run_finished` closes the measured flow after the target run completes.
- **Generate bootstrap, providers, classes:** `compilation_succeeded` and `compilation_failed` record generated-code compilation attempts. During runtime evolution, the same event types also measure the compile part of **Compile and redefine changed class**.
- **Unsupported input hits regenerate hook:** `method_intercepted` is emitted when the JDI breakpoint on a regenerate hook is hit. This event is the direct log evidence for an unsupported runtime scenario.
- **LLM agent writes implementation update:** `llm_request_succeeded`, `llm_request_failed`, and `code_generation_failed` record whether the agent call produced an extractable Kotlin update.
- **Compile and redefine changed class:** `generated_update_accepted` records that a generated implementation update was accepted for rebuild, while `hot_reload_succeeded` and `hot_reload_failed` record the JDI class-redefinition outcome.

The figure entries **Scan indexed Smart macro calls**, **Launch target JVM through JDI**, **Register breakpoints on regenerate hooks**, **Application reaches Smart macro call**, and **Resume original application call** describe necessary stages of the runtime flow, but they are not counted as separate validation events in the current log schema.

These logs let us reconstruct the runtime evolution loop after the run. *Run duration* is measured between `run_started` and `run_finished`. The number of unsupported runtime scenarios comes from `method_intercepted`. LLM request counts come from the `llm_request_*` events, while average LLM request duration is computed from their `durationMs` values. Compilation success rate is computed over LLM update attempts: requests that fail, fail code extraction, or produce non-compilable code count as unsuccessful attempts. Hot reload success rate is computed over compiled updates and shows whether the generated changes remained compatible with JVM class redefinition. The compilation and hot reload time shares are computed by summing the corresponding `durationMs` values and dividing them by the total run duration. Tool calls per LLM request are computed from the `count` field and measure how much project and runtime exploration the agent needed before submitting an update.

We use these metrics because they correspond to the main claims of the design. Compilation success rate evaluates the quality of generated code. Hot reload success rate evaluates whether the design keeps generated updates inside the constraints of JDI class redefinition. LLM request duration and tool-call counts describe the cost of using an agent rather than a single prompt. The accepted update count, derived from `generated_update_accepted`, shows how many times the Smart macro implementation successfully evolved during a run.

Plugin logs measure the mechanics of KotlinLLM, but they do not by themselves prove that the application result is useful. Therefore, each validation target also has a task-level metric. For Spring Petclinic Kotlin, the task-level metric is successful completion of all application scenarios after Smart macro evolution. For GitHub Issue Radar, the task-level metrics are precision and recall against ground-truth beginner labels. Recall is the primary metric for that benchmark because the application is a discovery tool: missing issues that are actually suitable for beginners is more damaging than returning extra candidates that can be filtered later.

9.2 Spring Petclinic Kotlin

The evaluation was conducted on our own version of Spring Petclinic Kotlin, derived from the open-source Spring Petclinic Kotlin repository [12]. The upstream project is a fully functioning Spring web server, but most of its request handling, response serialization, and Model-View-Controller behavior is delegated to Spring framework code. That made the original repository a useful realistic application base, but not a direct validation target for KotlinLLM, because too little application-owned Kotlin logic was available for replacement with Smart macros.

We therefore created an adapted Petclinic version specifically for this validation. The goal was not to change the Petclinic domain behavior, but to expose selected data-flow logic as ordinary Kotlin code and then replace that code with `asLlm` calls. In particular, we moved several conversions that were previously handled implicitly by Spring or by fixed application code into explicit application-level paths: reading and transforming database results, constructing route responses, and rendering the same domain data in JSON and XML formats. These exposed parts were replaced by 18 different calls to `asLlm`.

The app was tested by making calls to different routes with different scenarios, with 24 scenarios in total. All 24 scenarios completed successfully. During the tests, we observed how Smart macros updated the logic based on the different scenarios.

The validation tests used OpenAI GPT-5 as the driver LLM model for the KotlinLLM agent. The Spring Petclinic Kotlin run lasted 39m 48.6s. During this run, KotlinLLM tracked 18 `asLlm` call sites and completed all 24 application scenarios after Smart macro evolution. The run required 30 successful implementation updates. The rates below show the behavior more clearly: Smart macros recovered from unsupported scenarios, every hot reload succeeded, and compilation failures were contained inside the regeneration loop.

The agent did not always produce compilable Kotlin on the first attempt, but every compilation failure was repaired by a later request. This means that the correctness result depends not only on a single LLM response, but on the regenerate-compile-retry loop.

Metric	Value
<i>Run-level measurements</i>	
Run duration	39m 48.6s
LLM requests	36
Tool calls per LLM request	3.69
Average LLM request duration	64.9s
<i>Evolution outcomes</i>	
Compilation success rate	30 / 36 (83.3%)
Hot reload success rate	30 / 30 (100.0%)
Compilation share of run time	1.07%
Hot reload share of run time	0.08%

Table 1: Spring Petclinic Kotlin GPT-5 run summary

The 100% hot reload success rate is an important design signal. It verifies that Kotlin-LLM successfully restricts generated updates to changes that remain compatible with JDI class redefinition. The cost shares also show that compilation and class redefinition account for only a small part of the full run time.

9.3 GitHub Issue Radar

The GitHub Issue Radar task was evaluated using repository examples and ground-truth beginner labels from Awesome for Beginners [8]. The benchmark contains 20 repositories, 30,637 issues, and 311 beginner issues. This validation run also used GPT-5 as the driver model.

Label type	Repositories	Issues	Beginner %	Recall
Good first issue	10	23,018	0.92%	0.9384
Non-trivial	10	7,619	1.31%	0.7900
Overall	20	30,637	1.02%	0.8907

Table 2: GitHub Issue Radar benchmark results

Repository	Beginner label	Issues	Beginner %	Predicted	Ground truth	Precision	Recall
hyperium/hyper	E-easy Status: Good First	252	7.94%	20	20	1.0000	1.0000
chef/chef	Issue	369	1.08%	6	4	0.6667	1.0000
python/mypy	good first issue	3,143	0.29%	24	9	0.3750	1.0000
fabric/fabric	Low-hanging-fruit	500	0.80%	4	4	1.0000	1.0000
containerd/containerd	exp/beginner	587	0.51%	0	3	0.0000	0.0000
kubernetes/kubernetes	good first issue	2,700	0.37%	0	10	0.0000	0.0000
lxc/lxd	easy	79	10.13%	26	8	0.3077	1.0000
layer5io/meshery	good first issue	1,368	3.73%	51	51	1.0000	1.0000
moby/moby	exp/beginner	3,783	0.95%	50	36	0.6800	0.9444
pingcap/tidb	good first issue	6,233	0.98%	214	61	0.2850	1.0000
kashifkhan0771/utils	good first issue	2	50.00%	1	1	1.0000	1.0000
hasura/graphql-engine	good first issue	2,363	0.51%	33	12	0.3636	1.0000
CatimaLoyalty/Android	good first issue	167	2.99%	26	5	0.1923	1.0000
elastic/elasticsearch	good first issue Good-for-New-	5,704	0.49%	28	28	1.0000	1.0000
emberjs/ember.js	Contributors	379	1.06%	14	4	0.2857	1.0000
sindresorhus/eslint-p	good-for-beginner	373	1.34%	195	5	0.0256	1.0000
lugin-unicorn							
JabRef/jabref	good first issue	511	2.94%	25	15	0.4800	0.8000
questdb/questdb	Good first issue new contributor	827	2.30%	25	19	0.7600	1.0000
openstreetmap/iD	opportunity	1,068	1.50%	0	16	0.0000	0.0000

Table 3: GitHub Issue Radar per-repository results

The aggregate table includes all 20 evaluated repositories. The benchmark is intentionally sparse: beginner issues account for about 1% of all issues. The model is given time to inspect repository-specific label conventions. The overall recall of 0.8907 indicates that the generated logic found most of the ground-truth beginner issues, while the lower recall on non-trivial labels shows that repository-specific naming remains the difficult part of the task. It is worth noting that, for `containerd/containerd` and `kubernetes/kubernetes`, the model correctly identified the repository-specific beginner label, but failed to create a valid parser that iterates over the issues.

Precision is lower in this run, especially in repositories where the generated parser accepted broad labels such as `help wanted`, `enhancement`, or project-specific component labels. This shows that ranking and filtering the returned issues remains future work. It also exposes a flaw in the current LLM agent that needs further investigation: sometimes the generated guard is too vague. Once such a broad guard is added, it can affect all future cases handled by the same Smart macro. For example, the model might generate a case that only checks whether an issue has a label containing the word `easy`. This is why the `hint` parameter in `asLLM` is useful: it forces the model to distinguish scenarios using the developer-provided intent, instead of relying only on broad surface patterns in the runtime input.

Metric	Value
Run duration	1h 2m 49.5s
Total LLM calls	28
Compilation success rate	20 / 28 (71.4%)
Tool calls per LLM request	11.14
Average LLM request duration	114.7s
Hot reload success rate	20 / 20 (100.0%)
Compilation share of run time	0.81%
Hot reload share of run time	0.02%

Table 4: GitHub Issue Radar KotlinLLM run summary

10 Limitations

10.1 Platform Constraints

KotlinLLM is limited to JVM only, since the project depends on class reloading.

Since KotlinLLM uses JVM class redefinition, generated classes must already exist before the target application loads them. This prevents the system from freely adding new fields, methods, or classes after launch.

10.2 LLM Reliability and Security

KotlinLLM relies on the validity of LLM output. The LLM does not guarantee that it will understand the developer's intent or the runtime input correctly. This is visible in Table 3, where the GitHub Issue Radar per-repository results show that the generated logic can fail on repository-specific inputs. The failures are especially relevant for large inputs: when thousands of issues are present, the agent may miss the structure it needs to iterate over the response, or it may infer a guard that is too broad.

Security is a concern since KotlinLLM injects LLM-generated code into a running application. The current implementation uses a heuristic safety check before accepting generated handler code. The check rejects common unsafe patterns:

- package and import declarations;
- new type declarations, annotations, and labeled returns;
- filesystem access, network access, process execution, and JVM exit;
- console and environment access;
- reflection, classloader access, threading, and coroutine operations;
- time, randomness, and UUID generation;
- IntelliJ project or PSI APIs.

This heuristic is intentionally simple. It clears the most common and easiest unsafe cases quickly, and it fits the runtime update loop because it does not require an additional model call. However, it is not a complete security proof. A stronger future approach is to use an LLM-as-a-judge step to inspect generated code semantically. This would likely improve

the quality of the safety check, but it introduces a clear speed versus quality tradeoff: every safety decision would become slower and would add another model-dependent component to the regeneration loop.

10.3 Launch Configuration

KotlinLLM currently supports IDEA Kotlin run configurations, where the plugin is responsible for launching the target JVM. This lets KotlinLLM generate the required source files, start the application under JDI [10], and install regenerate-hook breakpoints before the application executes Smart macro code.

Gradle-based run configurations are harder because Gradle is responsible for launching the application JVM [5]. Supporting this setup would require the plugin to attach to a JVM that Gradle starts. That attach-based design introduces races: KotlinLLM cannot guarantee that the generated files are produced before the application loads the relevant classes, or that the breakpoints are installed before the application reaches a regenerate hook.

10.4 Type and Mock Support

It is worth noting that a limitation exists when the developer requests a parser that is not possible or does not have a clear meaning in the application. KotlinLLM does not try to decide whether every `asLlm<F, T>` request makes sense. It assumes that the developer uses `asLlm` only for meaningful parsing or transformation tasks. For example, asking KotlinLLM to parse a `Boolean` into an `Issue` data class has no clear interpretation unless the application defines what such a conversion should mean.

A known issue is that Smart macros do not support generics. It is inefficient and difficult to correctly create generated code for calls like `asLlm<F, T>()` when `T` is generic.

Another limitation is specific to `mockLlm`. The current implementation can only mock user-defined interfaces. It cannot mock classes, and it does not target interfaces from external libraries. This restriction keeps generated mocks inside the project model that KotlinLLM can inspect and regenerate reliably.

11 Conclusion

This thesis addressed a gap in how LLMs are integrated into software systems. Existing LLM support for software engineering mostly focuses on development-time tasks such as code generation and program comprehension, while runtime use is usually treated as direct delegation to an LLM service or as an external agent workflow. KotlinLLM explores a different point in this design space: developers can interact with an LLM directly from Kotlin application code, and the result of that interaction becomes ordinary generated Kotlin source.

The proposed design centers on Smart macros. A Smart macro is an explicit Kotlin function call whose behavior is implemented by generated logic containers. These containers start from a compilable empty state, detect unsupported runtime cases through a stable regenerate path, and let the IntelliJ plugin intercept those cases while the application is running. The plugin then gives the LLM agent targeted runtime and type context, compiles the generated update, and redefines the affected JVM class.

The important property of this design is persistence. KotlinLLM does not only ask an LLM for an answer at runtime. It turns the answer into project source code that can be reused on later executions, inspected by developers, tested, distributed, and run without another LLM call for already learned scenarios. This makes the runtime adaptation explicit and reproducible while preserving the flexibility of learning from concrete runtime values.

We verified the approach through two Kotlin validation targets. Spring Petclinic Kotlin tested repeated Smart macro use in a larger application flow. Both evaluated driver models completed all application scenarios after runtime evolution. GitHub Issue Radar tested unstructured API response parsing against repository data and exposed both the usefulness and the limits of the generated logic. Across these experiments, the validation measured compilation success, hot reload success, request duration, and task-level correctness. The results show that persistent runtime evolution for compiled Kotlin applications is feasible, while also identifying clear limits around LLM reliability, safety, and JVM class redefinition.

12 References

- [1] Anthropic. *Claude Code Overview*. 2026. URL: <https://code.claude.com/docs/en/overview> (visited on 05/17/2026).
- [2] Ellie Y. Cheng et al. “Sharing State Between Prompts and Programs”. In: *International Conference on Learning Representations*. 2026. URL: <https://openreview.net/forum?id=E7ZZRnBQU7>.
- [3] Jayanaka L. Dantanarayana et al. “MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming”. In: (2024). DOI: [10.48550/arXiv.2405.08965](https://arxiv.org/abs/2405.08965). arXiv: [2405.08965 \[cs.PL\]](https://arxiv.org/abs/2405.08965). URL: <https://arxiv.org/abs/2405.08965>.
- [4] GitHub. *GitHub REST API Documentation*. 2026. URL: <https://docs.github.com/en/rest> (visited on 05/17/2026).
- [5] Gradle. *Gradle User Manual*. 2026. URL: <https://docs.gradle.org/current/userguide/userguide.html> (visited on 05/17/2026).

- [6] Xueyu Hu et al. “Leveraging Print Debugging to Improve Code Generation in Large Language Models”. In: (2024). DOI: [10.48550/arXiv.2401.05319](https://doi.org/10.48550/arXiv.2401.05319). arXiv: [2401.05319](https://arxiv.org/abs/2401.05319) [cs.CL]. URL: <https://arxiv.org/abs/2401.05319>.
- [7] JetBrains. *Program Structure Interface (PSI)*. 2024. URL: <https://plugins.jetbrains.com/docs/intellij/psi.html> (visited on 05/14/2026).
- [8] MunGell. *Awesome for Beginners*. 2026. URL: <https://github.com/MunGell/awesome-for-beginners> (visited on 05/16/2026).
- [9] OpenAI. *Codex*. 2026. URL: <https://openai.com/codex/> (visited on 05/17/2026).
- [10] Oracle. *Java Debug Interface*. 2026. URL: <https://docs.oracle.com/en/java/javase/24/docs/api/jdk.jdi/module-summary.html> (visited on 05/14/2026).
- [11] Aske Plaat et al. “Agentic Large Language Models, a survey”. In: *Journal of Artificial Intelligence Research* 84.29 (2025). DOI: [10.1613/jair.1.18675](https://doi.org/10.1613/jair.1.18675). arXiv: [2503.23037](https://arxiv.org/abs/2503.23037) [cs.AI]. URL: <https://arxiv.org/abs/2503.23037>.
- [12] Spring Petclinic. *Spring Petclinic Kotlin*. 2026. URL: <https://github.com/spring-petclinic/spring-petclinic-kotlin> (visited on 05/14/2026).
- [13] Zhensu Sun et al. “Towards Agentic Runtime Healing”. In: (2024). DOI: [10.48550/arXiv.2408.01055](https://doi.org/10.48550/arXiv.2408.01055). arXiv: [2408.01055](https://arxiv.org/abs/2408.01055) [cs.SE]. URL: <https://arxiv.org/abs/2408.01055>.
- [14] Quanjun Zhang, Chunrong Fang, Yang Xie, et al. “A survey on large language models for software engineering”. In: *Science China Information Sciences* 69.141102 (2026). DOI: [10.1007/s11432-025-4670-0](https://doi.org/10.1007/s11432-025-4670-0). URL: <https://link.springer.com/article/10.1007/s11432-025-4670-0>.